

Documentación

Proyecto :

ODDSPY



Índice

- 1-Introduccion**
- 2- Justificación del proyecto**
- 3- Aspectos generales**
- 4- Contenidos**
 - 4.1. Acceso a datos
 - 4.2. Base de datos
 - 4.3. Despliegue
 - 4.4. Interfaz gráfica
- 5- Análisis del proyecto**
- 6- Desarrollo e implementación**
- 7- Pruebas e incidencias**
- 8- Manual de usuario**
- 9- Preguntas frecuentes**
- 10- Conclusiones**
- 11- Referencias (bibliografía y web)**

1-Introduccion

Oddspy es una aplicación diseñada para ayudar a los usuarios interesados en el mundo de las apuestas a ganar dinero sin riesgo. Su principal función es identificar oportunidades conocidas como "Surebets" (apuestas seguras) y estrategias de "Bonus Liberators" (aprovechamiento de bonos y freebets).

A través del análisis de cuotas en diferentes casas de apuestas, Oddspy permite a los usuarios encontrar combinaciones rentables, de forma que, apostando en todos los posibles resultados, siempre se garantice un beneficio o, al menos, la reducción máxima del riesgo.

El proyecto se crea con la necesidad de crear una herramienta que automatice la búsqueda de estas oportunidades y ayude a los usuarios a maximizar sus beneficios minimizando sus pérdidas.

2-Justificacion

Me decante por realizar este proyecto ya que en el mercado hay pocas opciones accesibles para el público español que cubran este tema . Hay opciones, pero estas son extremadamente caras por lo que cualquier persona no podría permitírselo además si interfaz es compleja y puede resultar caótica para una persona que este empezando en este mundillo .

Por lo que pensé que sería una buen idea ofrecer una app con una interfaz más amigable al público general y no tan especializada para que cualquier persona que siguiera el tutorial pudiera entender cómo funciona esto y además no tuviera que dejarse un dineral para poder pagar una suscripción mensual de uno de mis competidores



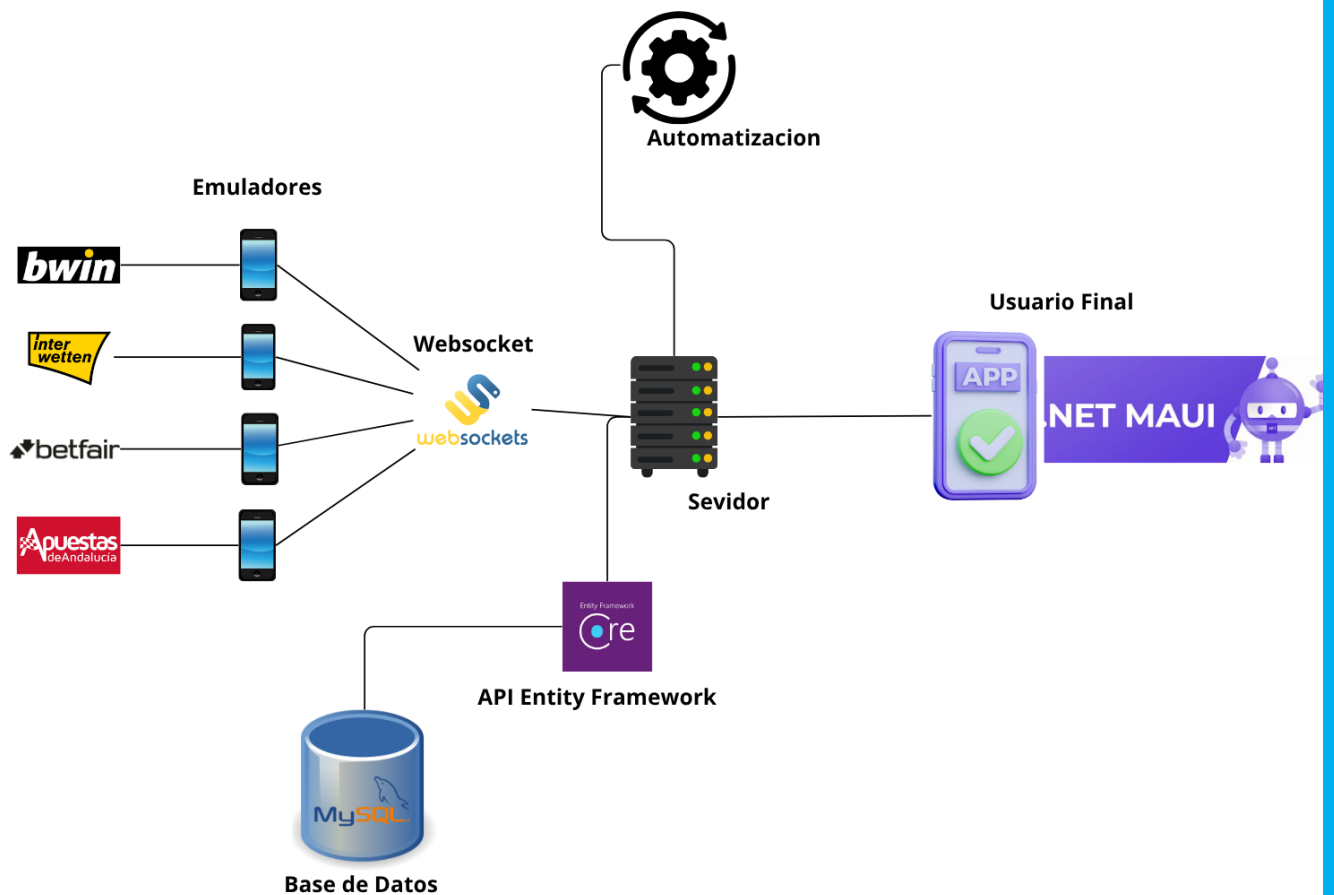
Precio de los competidores

3-Aspectos Generales

El funcionamiento de mi App se podría dividir en **3 partes** :

- Extracción de datos (Cyandroemu , pandas ,Virtual Box)
- Envío y procesamiento de los datos (Websocket Python)
- Interfaz que refleja todos los datos procesados (App Maui)

Mi app funciona de la siguiente forma :



Voy a explicarlo por encima cada una de las partes que lo componen :

Extracción de datos

Para la extracción de datos empleo la técnica conocida como webscrapping , para ello utilizo emuladores de Virtual Box con BlissOs(Android) y una librería de Python llamada Cyandroemu que lo que hace es poder descomponer los elementos de una app y así poder extraer los datos . Una vez he conseguido los datos “crudos” debo de limpiarlos de datos vacíos o que no me interesen y esto lo hago a través de pandas. Cuando todos estos datos han sido extraídos y limpiados se envían al websocket para que continúe su proceso

Envío y procesamiento de los datos :

Para el procesamiento y envío de los datos empleo la librería websocket y async de Python , con este web socket establezco la conexión con cada uno de los emuladores , una vez la conexión se ha establecido el emulador comienza a enviar los datos extraídos de la web , estos datos son procesados y se estandarizan los nombres de los distintos equipos ya que son diferentes en cada parte y se clasifican según sean surebets o bonnus liberators . Cuando un cliente se quiera conectar al websocket enviara la información dependiendo de los datos que se le solicite (surebet o bonnus liberators)

Interfaz que refleja todos los datos procesados

Para mostrar los datos obtenidos me decante por emplear Maui que es una Plataforma de .NET que permite crear aplicaciones multiplataforma . Aquí cuento con varias pantallas propias de una aplicación (Pagina Bienvenida , Loguin , Registro , Surebets , Comparaciones ...) para realizar el Loguin y la conexión a la base de datos decidí crear una API con Entity Framework que es la encargada de extraer la información de la DB , aquí se guardan las credenciales de los usuarios y demás datos del usuario . Además de las ofertas a las que está suscrito y las apuestas que ha realizado . Los datos se muestran a través de collections views en los que se puede ver las distintas apuestas que hay en el momento

Automatización

Toda esta arquitectura esta automatice en su mayoría, es decir , al arrancar el servidor hay servicios de Linux que corren cada uno de los elementos necesarios para el funcionamiento de la app . Hay un servicio que se asegura que el websocket está corriendo , otro que inicia las máquinas virtuales al encender el pc , otro que se encarga de ejecutar los scripts una vez la máquina virtual esta iniciada ...

Además, también el websocket y la API están expuestos a internet a través de un dominio "oddspy.store" esto nos permitirá acceder a los datos como si estuviera en producción . Tanto la API como el Websocket se ejecutan en un Puerto distinto y utilizan SSL gracias a certbot , todo esto está expuesto a través de Nginx

4-Contenidos

4.1 Acceso a Datos

En mi Proyecto lo que empleo para el acceso a datos es una Api desarrollada en Entity Framework de .NET , a través de aquí hago todas las consultas a la base de datos desde la app . Las principales funciones de esta api son:

- Registrar usuarios y autenticarlos.
- Mostrar ofertas y realizar operaciones en sus importes
- Registrar apuestas y gestionar la liberada de créditos

Esta API cuenta con 3 modelos principales (usuarios , ofertas y bets) que son relacionados por (UserOffer) .Además estos modelos son gestionados por 2 controladores distintos cada uno que son los encargados de exponer los endpoints .

Uno se encarga de las operaciones de usuario (Registro, Inicio de sesión , Obtener usuario por ID ...) y el otro gestiona (suscripción a ofertas , obtención de ofertas , liberación de créditos , En mi aplicación solo puede haber un tipo de usuarios por lo tanto el modelo de usuarios es bastante sencillo

```

public class User
{
    1 referencia
    public Guid Id { get; set; }
    4 referencias
    public required string Username { get; set; }
    5 referencias
    public required string Email { get; set; }
    4 referencias
    public required string Password { get; set; }

    1 referencia
    public decimal FreeBets { get; set; }

    1 referencia
    public ICollection<UserOffer> UserOffers { get; set; }
}

```

Modelo : User

```

public class Offer
{
    3 referencias
    public Guid Id { get; set; }
    3 referencias
    public string Casino { get; set; }
    3 referencias
    public string BannerImg { get; set; }
    3 referencias
    public string Description { get; set; }
    3 referencias
    public string Title { get; set; }
    5 referencias
    public int MaxFreeAmount { get; set; }
    3 referencias
    public string Type { get; set; }

    1 referencia
    public ICollection<UserOffer> UserOffers { get; set; } = new List<UserOffer>();
}

```

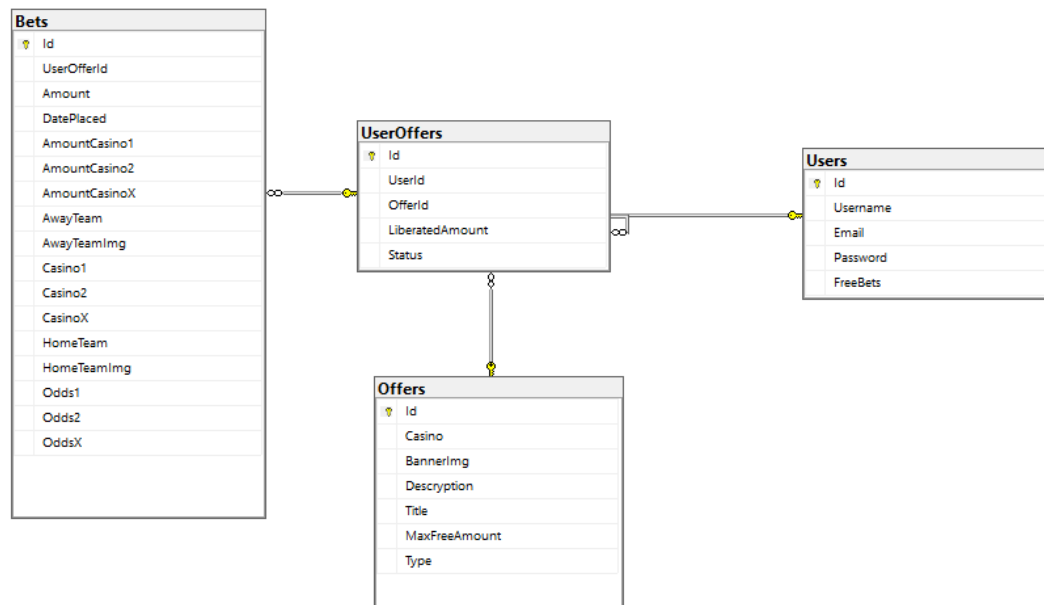
Modelo : Offer

Offers			^
GET	/api/Offers		▼
POST	/api/Offers/register		▼
POST	/api/Offers/registerBet		▼
GET	/api/Offers/userProgress/{userId}		▼
GET	/api/Offers/isRegistered		▼
GET	/api/Offers/getUserOfferId		▼
GET	/api/Offers/{id}		▼
GET	/api/Offers/userOffer		▼
GET	/api/Offers/userBets/{userId}		▼
User			^
GET	/api/User		▼
POST	/api/User		▼
GET	/api/User/{id}		▼
PUT	/api/User/{id}		▼
POST	/api/User/login		▼
PUT	/api/User/{id}/change-password		▼

Endpoints de la API

4.2 Base de datos

Como base de datos empleo MySQL y como gestor de base de datos empleo MySQL Server ya que es el gestor que emplea por defecto .NET y al tener experiencia con MySQL me decante por usarlo. Mi base de datos tiene el siguiente esquema entidad relación :



4.3 Despliegue

El despliegue se ha llevado a cabo en un servidor Ubuntu propio el cual se encarga de alojar todos los servicios (websocket y api) además de que tiene todo su funcionamiento automatizado usando servicios de Linux para que justo al encenderse vayan iniciándose todos los procesos .

Me decante por usar un servidor propio ya que para mí era más sencillo además de que para mi app se necesita un servidor bastante demandante respecto a hardware por lo que ningún servidor en la nube podría correrlo ya que al ejecutar varias maquinas virtuales necesita gran cantidad de memoria RAM y procesador . Las especificaciones de hardware de mi servidor son las siguientes:

- GTX 1060 6GB
- 32 Gb RAM DDR4
- 2 procesadores Xeon 5650
- Disco Duro de 1 TB ssd

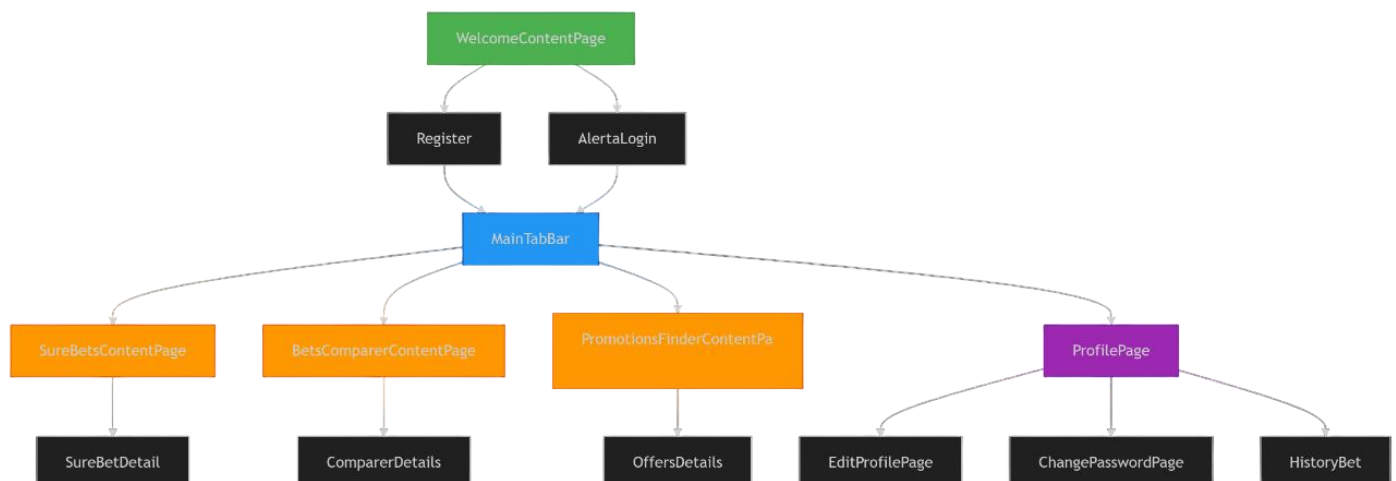
En el apartado de desarrollo de la app se explica con mas detalle como se ha realizado la automatización del despliegue y más características

4.3 Interfaz Gráfica

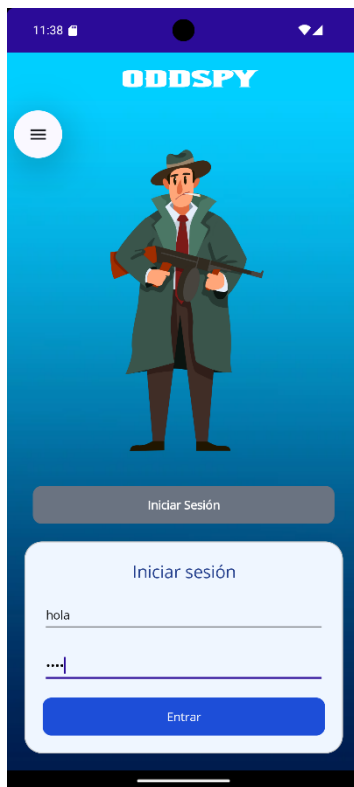
La parte de la interfaz gráfica la he realizado con MAUI . Por lo tanto, todos las vistas de mi aplicación están hechas con xaml . Mi aplicación se compone de las siguientes ventanas :

- **WelcomeContentPage (Pantalla de inicio)**
 - | - Register (Registro)
 - | - AlertaLogin (Inicio de sesión)
- **SureBetsContentPage (Surebets)**
 - | - SureBetDetail (Detalles de surebet)
- **BetsComparerContentPage (Comparador de apuestas)**
 - | - ComparerDetails (Detalles de comparación)
- **PromotionsFinderContentPage (Buscador de promociones)**
 - | - OffersDetails (Detalles de oferta)
- **ProfilePage (Perfil de usuario)**
 - | - EditProfilePage (Editar perfil)
 - | - ChangePasswordPage (Cambiar contraseña)
 - | - HistoryBet (Historial de apuestas)

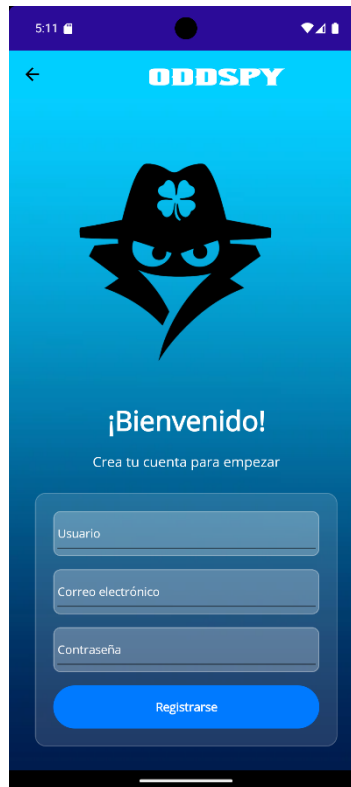
Para verlo más claro aquí adjunto el esquema de navegación :



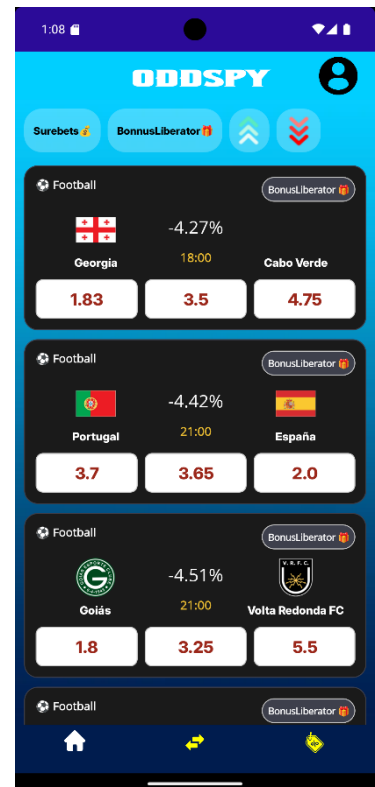
Y captura de algunas de las distintas pantallas de mi app :



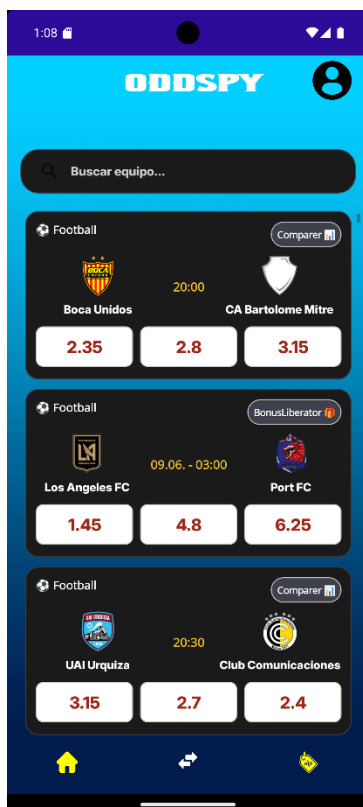
WelcomeContentPage



Register



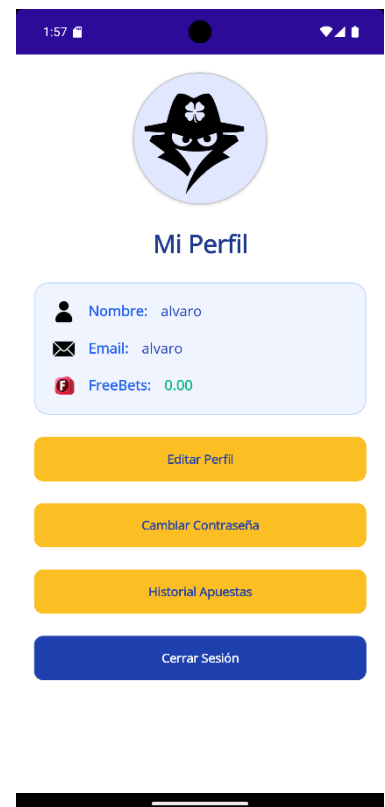
SurebetContentPage



ComparerContentPage



PromotionFinderContentPage



ProfilePage

5-Analisis del Proyecto

Mi consiste en una app que se encarga de ayudar al público que le gusten y esté interesado por el tema de las apuestas a poder conseguir dinero sin riesgo . Esta app se encarga de encontrar apuestas en las cuales no se puede perder nunca denominadas “Surebets”, Y dirán: ¿ Como es posible realizar una apuesta en la que nunca se pierda ? ! ¡Pues muy fácil !

Apostando a todos los resultados posibles de un partido , es decir supongamos que juega Real Madrid vs Barca y sus cuotas son tal que así :

Real Madrid : 3.9 Empate: 3 Barca : 2.1

Si apostásemos a todos los resultados con estas cuotas veríamos que no podemos ganar dinero de hecho perderíamos ya que los casinos ganan dinero gracias a lo que se llama “Edge” que es el margen que tienen para asegurarse de que ganan siempre esto se puede calcular de manera muy sencilla :

$$1/3.9*100 + 1/3*100 + 1/2.1*100 = 106.59\%$$

En este caso es del 6.59% que se podría considerar un “Edge promedio” , la gracia de mi aplicación es que compara cuotas entre distintos casino para intentar bajar el Edge , encontrado las cuotas más altas posibles . Cuando hacemos esa fórmula y nos da por debajo de 100% quiere decir que ya es rentable para nosotros ya que si le apostamos a todos los resultados Ganaremos SI o SI . Lo malo de esto es que no suelen ser muy comunes encontrarlas y se suelen localizar más frecuentemente en épocas en las que hay gran afluencia de partidos en el mundo por lo que es difícil encontrarlas “fuera de temporada” .

Pero no nos debemos de preocupar por eso ya que no es la única forma de ganar dinero sin perder nada . Oddspy nos proporciona otra forma que es empleando las apuestas “Bonnus Liberators” estas apuestas son apuestas con un Edge bajo lo que nos permite aprovechar bonos de bienvenida y promociones de Freebets para poder sacarles rentabilidad . Pongo un ejemplo para que sea más entendible :

Carlos Alcaraz : 1.15 Lorenzo Musetti : 6.5

$$1/1.15*100+1/6.5*100 = 102.29\%$$

Aquí como vemos el Edge es de solamente un 2.29% por lo que se puede considerar bajo , ahora imaginemos que nos acabamos de registrar en un casino que tiene un bono de bienvenida de 100€ si perdemos esos 100€ nos los devuelven en créditos de apuesta .

Nuestra intención es intentar perder ese dinero ¡No te asustes! Ese dinero se ganará en otra casa de apuestas por lo que no perderás nada a parte del "Edge" .

La mejor estrategia para intentar perder el dinero y así nos den las preciadas freebets (Si ganamos normalmente no nos las suelen dar, aunque depende de la promoción y del casino) es apostar a un partido en el que haya una cuota muy alta por lo que sea muy poco probable que pase y apostar los 100 € a que gana el equipo con cuota más alta . Esta es la forma más fácil para poder convertir los 100€ a freebets . En nuestro caso haríamos lo siguiente :

-Apostar 100€ a **Lorenzo Musetti** : $100 \times 6.5 = 650\text{€}$

-Apostar 650€ a **Carlos Alcaraz**: $565 \times 1.15 = 649.75\text{€}$

Como vemos podemos Balancear la apuesta para que nunca perdamos nada más del Edge en nuestro caso como es el 2.29% de 650 que corresponde a **15,24€** .Esa sería nuestra perdida a cambio de conseguir 100€ en freebets .

Por cierto, la cantidad que hay que apostar a cada cuota te la calcula automáticamente al introducir la cantidad Oddspy .

El segundo paso para liberar nuestra oferta seria volver a realizar otra apuesta igual solo que empleando las freebets , ahora hemos encontrado incluso una mejor cuota

Novak Djcovich : 1.10 C. Norrie : 10

$$1/1.10 \times 100 + 1/10 \times 100 = 100.9\%$$

Ahora el Edge es de solo un 0.9% esto es muy bueno ya que apenas perderemos nada . Volvemos a distribuir las apuestas

-Apostar 100€ de Freebets a **C. Norrie** : $100 \times 10 = 1000\text{€}$

-Apostar 910€ a **Novak Djcovich**: $910 \times 1.10 = 1001\text{€}$

El Edge en este caso sumaria 9€ de perdida por lo que en total de beneficio habríamos obtenido :

$$100 - (15,24 + 9) = 75,76\text{€}$$

75.76€ sería el rendimiento obtenido de los 100€ de freebets que nos regalaron.

También hay formas de tener que apostar menos dinero ya que no todo el mundo quizás quiera o pueda apostar 900€ , el caso es que cuanto más alta sea la cuota y más será la probabilidad de que nos den las freebets . Si no quisiéramos apostar tanto dinero nos la podríamos jugar a una cuota más equiparadas para que no tuviera que ser el monto de dinero tan grande por ejemplo una cuota 2.5 y otra 1.6. Eso si aquí correríamos mucho más riesgo de no Ganar nada incluso perder la parte del Edge.

Esto puede mejorar si por ejemplo nos den como requisito de la oferta que apostemos los 100€ ingresados .Esto nos beneficia mucho ya que no necesitamos que necesariamente pierda para que nos las den por lo que son la mejor promoción de bienvenida que vamos a poder encontrar . De esta forma no necesitaríamos apostar tanto dinero y la pérdida del Edge sería mucho más baja (Al apostar menos cantidad)

6-Desarrollo e implementación

El desarrollo lo voy a dividir como hice anteriormente solo que ahora los explicare de forma más detallada :

- Extracción de datos
- Envío y procesamiento de los datos :
- Aplicación
- Scripts de Automatización

Extracción de datos



Esta fue la primera parte con la que comencé mi Proyecto . Al principio no tenía una idea muy clara de cómo poder extraer la información de las webs de los casinos , yo había utilizado selenium que es una librería para hacer webscrapping . Pero con un poco de búsqueda me di cuenta que iba a ser complicado encontrar un método que funcionara sin ser bloqueado .

La mayoría de formas convencionales de extraer datos a través de webscrapping consisten en utilizar alguna de las siguientes librerías (playright , puppeteer o selenium) estas librerías lo que hacen es extraer el HTML y poder filtrar a través de los componentes de la web .

El problema que tiene esto es que la mayoría de los casinos conocen de su existencia por lo que suelen detectarlos de forma fácil y bloquean la ip desde la que se está haciendo el scrapping además mucha de la información de los casinos no se puede obtener por HTML ya que mucha de ella se carga a través de scripts de JavaScript por lo que lo hacen imposible de obtener para este método .

Por lo tanto, debía de buscar una alternativa que no se pudiera detectar fácilmente y que obtuviera los datos de forma constante y sin problemas . Después de muchas búsquedas conseguí encontrar una librería escrita en Python llamada Cyandroemu .

Esta librería nos permite descomponer los elementos de cualquier app de Android a través del empleo de uiautomator , También Podemos controlar el dispositivo Android a nuestro antojo gracias a la automatización implícita de Android (ADB) .

Este método es infalible ya que no se puede detectar nada por parte de la web que visitamos , con Cyandroemu descomponemos los componentes que está mostrando el navegador y una vez descompuestos filtro los datos y los extraigo limpios . Esta forma nunca podrá ser detectada ya que nosotros simplemente estamos accediendo a una web como si fuéramos un usuario legítimo y no interactuaremos de ninguna forma con ella y aunque lo hiciéramos pareceríamos un usuario legítimo ya que emplearíamos el propio adb del dispositivo implementado en Cyandroemu. También hay que decir que el dispositivo Android es capaz de correr el código Python gracias a que está rooteado y tengo acceso al terminal a través de termux . Esto me permite ejecutar código dentro del propio dispositivo Android .

Una vez sabiendo esto me puse manos a la obra el primer paso era instalar la imagen del emulador para ello use BlissOs que es una versión de Android , utilice este Sistema ya que era el que menos recursos necesitaba y mejor funcionaba de acuerdo con la documentación de la librería . También lo use ya que este Sistema al ser rooteado no es detectado por ningún programa de detección de root del dispositivo (RootBeer) y esto hace aún más difícil que podamos ser restringidos.

Una vez instalado blissOs en VirtualBox seguí la documentación de la librería para poder instalar todos los paquetes necesarios para el correcto funcionamiento . He de decir que cada una de las máquinas virtuales solo funciona en el equipo que se han compilado todas las librerías ya que muchas de estas librerías incluida Cyandroemu son compiladas en C y Python y esto hace que al cambiar de equipo por lo tanto de procesador se necesite compilar de nuevo el código para su funcionamiento .

Esto no es un gran problema ya que se soluciona desinstalando las librerías que requieren compilación en C y volviéndolas a instalar en el nuevo dispositivo .

Una vez tenía todo esto configurado comencé a trabajar en cada uno de los scripts para extraer los datos de cada uno de los casinos . Yo como prueba use 4 casinos, pero esto se podría aumentar a más. Eso si teniendo en cuenta que se necesita un servidor dedicado para este propósito.(1 casino = 1 emulador)

Emplee estos 4 casinos :

-BetFair

-Bwin

-Interwetten

-Apuestas de Andalucía

El desglose del funcionamiento de los scripts es el siguiente :

El script se compone de una parte común para todos los scripts que es la que extrae los datos del navegador y la parte que los envía al websocket .

Y luego tiene la parte personalizada para cada script en la cual se filtra dependiendo de la forma en la que vengan estructurados los datos en el dataframe de pandas y como se distribuyan los distintos formatos de cada datos (fechas , nombre , orden de datos ...)

Por lo general para filtrar los datos se emplean filtros de pandas y alguna expresión de regex para poder comprobar algunos formatos más complejos . También se hace uso de los arrays de numpy para poder comprobar y extraer la estructura de los partidos y descartar datos que se hayan podido meter entre medias de estos .

Esto sigue unas condiciones booleanas en las que se comprueba si los datos de cada fila siguen un cierto formato(partido , fecha , cuota) , si es así se añade a otro dataframe de pandas para poder visualizar los datos mejor y poder enviarlos .

Una vez se han extraído los datos se llama a la función del websocket que hay en el propio script de cada casino y envía los datos a través del Puerto 8765 al websocket en formato json.

Envió y análisis(Websocket)



Para el envío y el análisis de los datos, decidí emplear la librería websocket de Python. Decidí que era imprescindible el uso de un websocket ya que los datos de los partidos cambian constantemente por lo que necesitaba una solución que fuera capaz de transferir los datos lo más rápido posible ya que de lo contrario podríamos perder alguna oportunidad de surebet o alguna otra apuesta valiosa.

El websocket está segmentado en varios archivos para su mejor mantenimiento y funciona de la siguiente forma:

Websocket

- |— Common.py
- |
- |— Main.py
- |
- |— Reciver.py
 - |— reciver_handler.py
- |
- |— Sender.py
- |
- |— ProcesadorApuestas.py
- |
- |— nombres_no_encontrados.py
- |
- |— equipos_estandar.json

Common.py

Este script almacena los mensajes entrantes de los emuladores en una cola en la que serán procesados más tarde , además También guarda los clientes que se han conectado por tipo (Surebets , bonus liberators , ofertas) y almacena las conexiones de los emuladores

Main.py

Este script se encarga de ejecutar todos los demás de forma organizada y de forma asíncrona . Las funciones que inicia son : Iniciar el websocket , Escuchar los receptores y Enviar los mensajes

Reciver.py

Se encargar de establecer la conexión inicial con los emuladores y añade los mensajes recibidos a la cola de procesamiento

Reciver_Handler.py

Este script se encarga de gestionar las conexiones con los clientes y gestionar el tipo de datos que quieren recibir (Surebets , ofertas , bonus liberados) dependiendo del tipo de dato que solicite se le enviara una cosa u otras .

ProcesadorDeApuestas.py

Se encarga de procesar todos los mensajes que son añadidos a la cola . Sigue el siguiente procedimiento para procesar los datos :

1º-Estandariza el nombre de los partidos con rapizzfuz que es una librería que nos permite hacer comparaciones entre strings en este caso yo cuento con un json en el cual hay el nombre de miles de equipos lo que hace el script es recorrer este json y busca a ver si hay alguna coincidencia o nombre parecido al equipo que se está estandarizando . Si lo hay lo estandariza y le agrega la imagen que se encuentra en el json . De no encontrarse en el json se añadirá a otro json llamado equipos no encontrados que hay es donde se guardaran todos los equipos cuyo nombre no esté en la lista para en un future poder añadirlos .

2º. Una vez estandarizados los nombres de los equipos se proceden a identificar si son Surebets, o bunnus liberator dependiendo del Edge que tengan. Para ello se comparan las cuotas de cada uno de los casinos recibidos que tengan el mismo Partido. Se cogen la cuota más alta de cada uno y se aplica la fórmula para ser clasificado.

Sender.py

Es el encargado de enviar los datos a los clientes en función del tipo de apuesta que soliciten y quieran visualizar. Aquí se llaman a las funciones de ProcesarApuesta.py y se guardan todos los partidos separados por tipo en un set cada uno así es más fácil compararlos ya que no hay duplicados y solo se queda el que mejor cuota tiene.

Este set se borra periódicamente para asegurarse de que se están actualizando los partidos procesados y no se está comparando con cuotas altas pero que ya no son válidas. Los partidos se envían en un Interval periódico que se puede configurar y además se formatea el json que va a ser enviado para la app.

Aplicación(MAUI)



La aplicación esta desarrollada en Maui ya que me pareció interesante la idea de realizar una aplicación multiplataforma escribiendo un solo código. La aplicación se compone de las siguientes ventanas:

WelcomeContentPage

Es la pestaña de bienvenida de la app, contiene los accesos a la Ventana de registro y a la de inicio de sesión. Simplemente realiza una petición a la api con los datos que el usuario ha introducido y si son correctos los redirige a la siguiente página que es SurebetsContentPage. Y guardamos el id de usuario para realizar más operaciones posteriormente.

Register

Es la pestaña como es obvio se gestiona el registro de nuevos usuarios , aquí se hace una petición a la api con los datos que ha introducido el usuario y se crea en la BD .

SureBetContentPage

Aquí se muestran las distintas apuestas disponibles tanto Surebets , como bonus liberados. Además, nos permite filtrar en orden ascendente y descendente la cuotas y También nos permite clasificar entre Surebets y bonus liberados . El código de esta página se encarga de mostrar un recycle view el cual lo obtiene de deserealizar el json que recibe del websocket y obtiene la conexión con el indicándole que es un receptor que quiere el tipo de ofertas Surebets .

También mantiene ciertos mecanismos para evitar que el collection view vaya con lag y que la conexión del websocket sea correcta .Además de esto se encarga También de pasar los datos de cada partido a la siguiente Ventana que es SureBetsDetails.

SureBetDetail

Aquí Podemos ver los datos concretos de cada uno de los partidos (Fecha , cuotas , casino ...) además aquí témenos la opción de seleccionar oferta si tenemos alguna disponible en función si nos hemos suscrito a ella o no .También cuenta con una calculadora de apuestas para permitirnos dividir nuestro dinero entro las distintas cuotas .

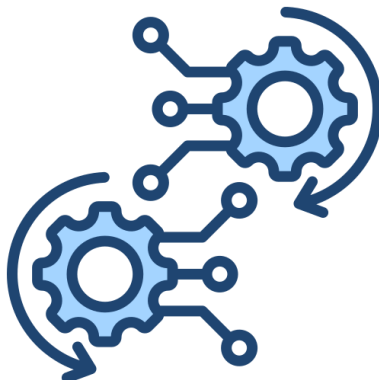
PromotionFinderContentPage

En este apartado se muestran ofertas de bienvenida que constan en la base de datos , por lo que simplemente realizamos una petición a la api y nos rellena el collection view con la respuesta que ha recibido de la BD .

OffersDetails

Se muestran las distintas condiciones de cada una de las ofertas que hay ,y También nos da la opción de indicar si en la realidad estas suscrito a ellas o no . Para ello cuenta con un botón el cual pone inscribirse y al hacer clic hace una llamada a la api en la cual usa el id del usuario y el id de la oferta sobre la que se ha hecho clic para añadirlo a la table que relaciona ambas y así saber que el usuario esta registrado en esa oferta . Además, También se puede ver una barra de progreso en la cual se muestra la cantidad de dinero que nos falta por apostar para poder liberar las freebets.

Scripts Automatización



En este apartado voy a explicar el Desarrollo de todos aquellos scripts que permiten al servidor ejecutar todo de forma automática y sin mi intervención . Simplemente voy a explicar aquellos scripts que sean para la propia automatización ya que hay varios que simplemente han sido establecidos como servicios de Ubuntu y cada vez que se inicia el Sistema se Lanza el servicio y permanece corriendo todo el rato . Los scripts que son solo servicios son :

- Iniciar Máquinas Virtuales : Simplemente es un script que se encarga de iniciar las máquinas virtuales que hay en el servidor al encenderse
- Iniciar Websocket : Inicia el websocket al encenderse el servidor
- Iniciar Api : Inicia la API al arrancar

El resto de servicios dependen de otros scripts que sí que merece la pena explicar y que están relacionados con ejecutar el código dentro de las máquinas virtuales y asegurarnos su correcto funcionamiento .

GenericStart.py

Cada una de las máquinas virtuales cuenta con un script como este personalizado . Su principal función es situar al emulador en la página del casino que le corresponde.

Empleando Cyandroemu al iniciar la maquina hago que se haga clic en el acceso directo del navegador y busque la url del casino que le corresponde scrapear . Además, dependiendo del casino activa la vista de escritorio para poder recopilar mejor los datos o navega a través de la página del casino para llegar a los eventos de futbol de hoy.

StartUDPVirtualBox.py

Intentando automatizar todo el código me di cuenta de un problema que me sucedía que era que necesita ejecutar los scripts de cada uno de los casinos que se sitúan dentro de las Máquinas virtuales . Como primera opción pensé en utilizar termux boot que es un terminal que al iniciarse tiene por defecto la función de ejecutar el comando que le introduzcamos dentro es como si fuera un cron Job de Linux que se ejecuta cada vez que se enciende el dispositivo .

Probe a hacerlo de esta forma y no dio resultado ya que al ser una distribución de Android no podía ejecutar directamente comandos como administrador dentro de termux boot por lo que no podía ejecutar el script para scraperar los casinos .

Por fin se me ocurrió una solución para solventar esto y era que en vez de ejecutar directamente el script con termux boot , usaría a termux boot para enviar un mensaje UDP al servidor con la palabra “actívame” como enviar mensajes UDP con netcat no require ser administrador esto funcionaba de maravilla .

Es decir, cuando una máquina virtual se enciende esta lo que hace es mandar el mensaje de UDP “activar” al servidor , en el cual hay un Puerto configurado que es el 9999 para escuchar las solicitudes . Al recibir el mensaje el servidor se conecta por ADB a la ip desde la cual ha recibido el mensaje y lo ejecuta . Ya que desde adb sí que tengo permisos de admin para poder ejecutar Scripts como root .

Mas tarde también le implemente una opción de reiniciar en el caso de que sé que crasheara el navegador y no tuviera remedio más que el reinicio , además cuenta con otro método que es que si se queda en la pantalla de inicio Vuelve a ejecutarse el script de abrir el navegador y comenzar a scrapear.

StartScript.py

Es el encargado de ejecutar GenericStart y una vez se ha terminado de ejecutar este , lanza el propio script de Scrapping del Casino . Este es el script que llama StartUDPVirtualBox al recibir el mensaje de activar .

Dominio



Tanto el websocket como la api están expuestos en el dominio “oddspy.store” . El servidor se encuentra en una lan con una ip fija por lo tanto no puede cambiar nunca por el DHCP . También está configurado un DNS dinámico para que en caso de que cambie la dirección IP publica se siga pudiendo acceder a el dominio.

La exposición de los distintos puertos la he hecho a través de Nginx y he empleado Certbot para crearles certificados firmados seguros de Lets Encrypt y que se renuevan de forma automatica gracias a certbot.

7-Pruebas e Incidencias

Pruebas

Durante el Desarrollo del Proyecto me he visto obligado a hacer muchos elementos a fuerza de prueba y error como es el caso de la extracción de los datos de los casinos que fue realizada así ya que de otra forma hubiera sido imposible debido a que hay que ir encontrando las distintas similitudes y diferencias de los datos deseados respecto a los demás .

He de decir que todos los endpoints de la API han sido testeados con postman antes de su funcionamiento

También todo el apartado de scripts de automatización teniendo en cuenta que debía de probar si todo se ejecutaba de forma fluida y sin errores .

Y otra parte en la que ha sido imprescindible las pruebas ha sido en el desarrollo de la interfaz de la app ya que al contrario que Android Studio no hay un interfaz en la cual te permita diseñar en Maui la vista de la aplicación por lo que tienes que hacerlo tu a mano en el xaml .

Entonces para comprobar que se viera la interfaz como yo quería era necesario ir haciendo pruebas de uno en uno para ver qué cambios quedaban mejor que otros

Incidencias

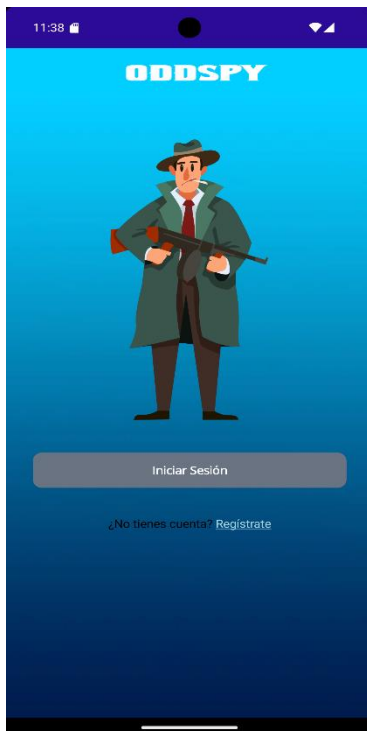
Las principales incidencias que he tenido con mi Proyecto han sido 2 :

-Encontrar Una forma de hacerle Scraping a los casinos sin ser detectado

-Solventar el problema de falta de permisos de StartUDPVirtualBox

-Hacer el sistema de la clasificación de los partidos y la distribución en sets de cada uno ya que de no hacerlo así se sobre escribía cada uno de los mensajes que mandaba el servidor de websocket con uno nuevo y no le daba tiempo en la interfaz a ver ningún Partido ya que cambiaban constantemente

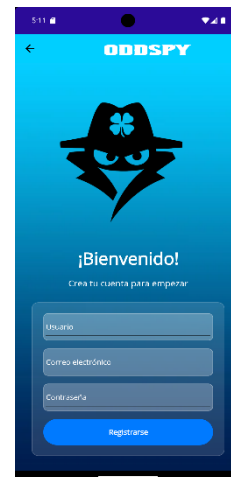
8-Manual de Usuario



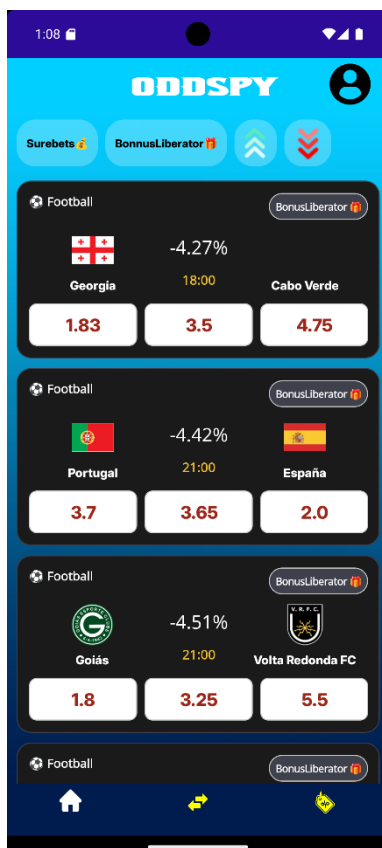
Los Primeros pasos con mi aplicación . Nada más abrirla te aparecerá la pantalla de Bienvenida en la que podremos ver un botón y un enlace estos nos servirán para poder hacer el inicio de sesión y en caso de no tener cuenta creárnosla .



Ventana de Loguin

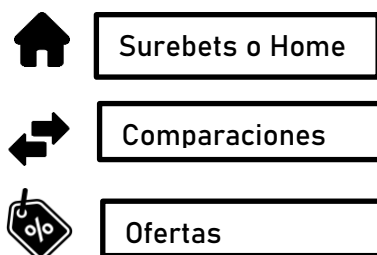


Ventana de Registro



Una vez nos hayamos o bien registrado o iniciado sesión , la aplicación nos conducirá a el menú principal de la app .

Aquí podremos navegar entre los distintos apartados que se encuentran en la barra de navegación inferior estos están representados por los símbolos :



En esta pestaña que estamos situados se nos muestran las Surebets o bonnus liberators que hay actualmente ,en la prevista nos muestra el porcentaje de perdida y las mejores cuotas a cada uno de los resultados .Además podemos ordenarlos en función de su tipo y su valor



Si hacemos clic encima de uno de los partidos veremos que nos aparece la siguiente pantalla . Aquí se nos muestra de forma mas detallada a que casino debemos de apostar cada resultado y que ofertas tenemos disponibles ya que al ser de tipo bonos liberator es el principal propósito .

Si bajamos un poco más abajo podremos ver una especie de calculadora en la cual nos indica que dinero debemos de apostar a cada una de las opciones . Este dinero apostado nos contara para liberar una oferta u otra teniendo en cuenta cual hayamos seleccionado. Una vez lleguemos al tope del progreso recibiremos las freebets que podremos verlas en el perfil .

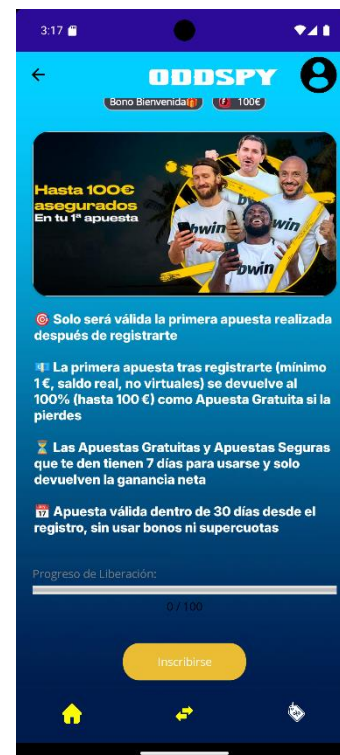
Hagamos un ejemplo de uso :

Imaginemos que quiero apostar a el partido de la captura para liberar mi bono , lo único que tendría que hacer seria seleccionar la oferta e introducir la cantidad que quiero apostar para ver como debo de distribuir mi dinero .

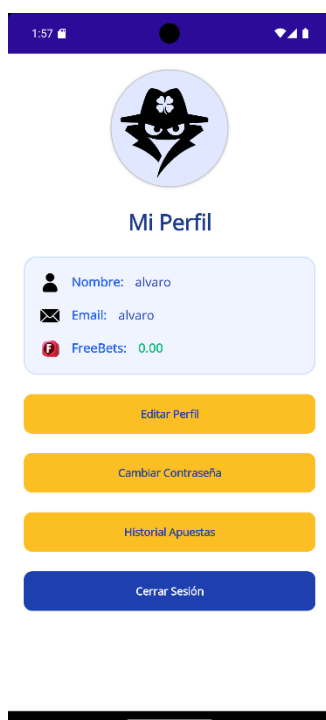
Una vez hecho esto le daremos al botón de aceptar y listo



Ahora si regresamos al menú principal y no desplazamos a el apartado de comparaciones podremos ver todos los partidos que detecta Oddspy , aquí la función que hace es encontrar la mejor cuota para cada partido, aunque no sirva para bonnus liberator ni surebet. Simplemente si un usuario esta interesado en sacarle el mayor rendimiento a su dinero puede ir a este apartado y buscar el partido que quiere apostarle . Una vez le haga clic encima se la abrirá la ventana de la derecha en la que muestra que casino tiene cada cuota .

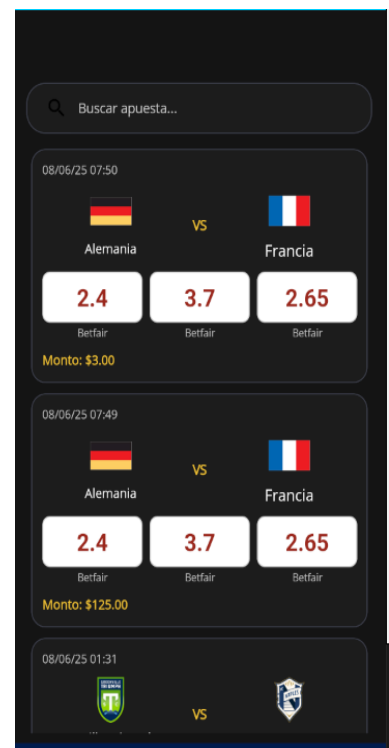


El último de los apartados de la barra de navegación es el de ofertas aquí el usuario podrá visualizar las ofertas disponibles que consigue obtener Oddspy y cada una de sus condiciones . Si hacemos clic encima nos mostrara la oferta de forma más detallada explicando sus características y peculiaridades . También se encuentra el botón de suscribirse para que el usuario pueda indicar si se ha registrado en uno de esos casinos y posea alguna de estas ofertas . Cuenta con una barra de progreso para visualizar cuanto dinero nos queda por liberar con las apuestas que hemos hecho



Por último, tenemos el apartado de el perfil del usuario en el cual puede ver sus distintos datos y cambiarlos a su antojo . Este apartado cuenta con otras 4 funciones que son :

- Editar Datos del perfil
- Cambiar la contraseña
- Ver el Historial de apuestas
- Cerrar la sesión actual



9-Preguntas frecuentes

¿La app de Oddspy hace apuestas por mí al darle al botón de apostar?

-No la app de Oddspy solo se encarga de llevar un seguimiento sobre las apuestas realizadas y mostrar aquellas que sean beneficiosas para el usuario

¿Qué deportes cubre Oddspy?

Actualmente, Oddspy se centra en partidos de fútbol, pero en futuras actualizaciones se planea incluir más deportes como baloncesto y tenis.

¿Por qué a veces no hay Surebets disponibles?

Las Surebets son oportunidades raras y dependen de las cuotas ofrecidas por los casinos. En temporadas con menos eventos deportivos, es más difícil encontrarlas. Sin embargo, siempre puedes aprovechar las estrategias de Bonus Liberators

¿Cómo sé si una oferta de casino es válida?

Oddspy muestra las condiciones de cada oferta en la sección "Promociones". Además, puedes marcar las ofertas a las que estás suscrito para llevar un seguimiento de tu progreso.

10-Conclusiones

-Pienso que en gran medida y dentro de mis posibilidades he cumplido bastante bien los objetivos que me puse al principio (Creer app para detector Surebets) . Ya que, aunque no es perfecta ni de lejos , funciona de forma correcta y consigue mostrar los datos de manera estable . Además, la gran mayoría de software que he empleado no lo conocía y nunca lo había dado por lo que considero que me ha servido de gran ayuda a seguir formándome en este mundo de la informática.

Como funcionalidades o contenidos que se le podrían añadir en el futuro y no serían muy difíciles de implementar, pero por falta de tiempo no se han implementado entrarían los siguientes .

-Expandir la detección de cuotas a más deportes (Baloncesto , Tenis ...)

-Entrenar una IA destilada gratuita como puede ser Deepseek para hacer la detección de datos erróneos en el filtrado de los datos de los partidos y así evitar tener que hacer un mantenimiento constante .

-Realizar Apuestas de forma automática en las webs de los casinos con Cyandroemu para no estar limitados por el tiempo tan corto que suelen durar las Surebets

-Ampliar la detección de ofertas y supercuotas a otros casinos

11-Bibliografía

Desarrollo API

<https://www.youtube.com/@SameerSaini>

Diseño de la Aplicación y demás Ventanas

<https://www.youtube.com/@abhayprince/videos>

<https://www.youtube.com/@jfversluis>

<https://www.youtube.com/@DanielHindrikes>

Extracción de datos

<https://github.com/hansalemaos/cyandroemu>

<https://pandas.pydata.org/docs/>

Websocket

<https://websockets.readthedocs.io/en/stable/>

<https://docs.python.org/3/library/asyncio.html>

https://www.youtube.com/watch?v=Vt4DJV45sKY&list=PLbCUoS9rATH6Z-36Hb2bL46mrc_3E3lFM&ab_channel=EmilyWebb

...